

Expense Tracker Implementation Report

Executive summary

This delivery implements the requested **Expense Tracker** as a full-stack CRUD application. The backend uses Django REST Framework with SQLite and exposes a REST API; the frontend uses React with a responsive personal-finance dashboard called **Ledger**.

Delivered functionality

Area	Implementation
Expense CRUD	Create, list, update, and delete operations
Search and filters	Search by title, notes, or category; filter by category
Summary	Grand total and totals grouped by category
Validation	Required title/date, positive amount, allowed category choices on client and server
Data model	Title, decimal amount, category, date, notes, created/updated timestamps
UI	Responsive sidebar, spend hero, expense form, activity list, category bars, empty/loading/error states
Integration	CORS configured for local React development; API URL configurable via <code>REACT_APP_API_URL</code>
Tests	API CRUD, summary, validation, and 404 coverage

Project files

- `backend/` contains Django settings, URL routing, the `expenses` app, model, serializer, viewset, admin registration, and tests.
- `frontend/` contains the React app, API client, dashboard UI, and responsive styles.

API endpoints

- GET/POST /api/expenses/
- GET/PATCH/PUT/DELETE /api/expenses/{id}/
- GET /api/expenses/summary/
- List supports `search` , `category` , `date` , and `ordering` query parameters through Django REST Framework configuration.

Run locally

Bash

```
cd backend
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py test
python manage.py runserver
```

In a second terminal:

Bash

```
cd frontend
npm install
npm start
```

The frontend defaults to `http://127.0.0.1:8000/api` . Set `REACT_APP_API_URL` in a `.env` file to point to another backend.

Verification

The backend test suite is designed to verify successful CRUD and summary behavior, invalid input rejection, and missing-resource 404 responses. The frontend can be checked with `npm run build` , which validates that the production React bundle compiles successfully.

Future enhancements

Authentication and per-user data ownership, monthly/yearly analytics, CSV export, budget alerts, and PostgreSQL deployment configuration are natural next steps.